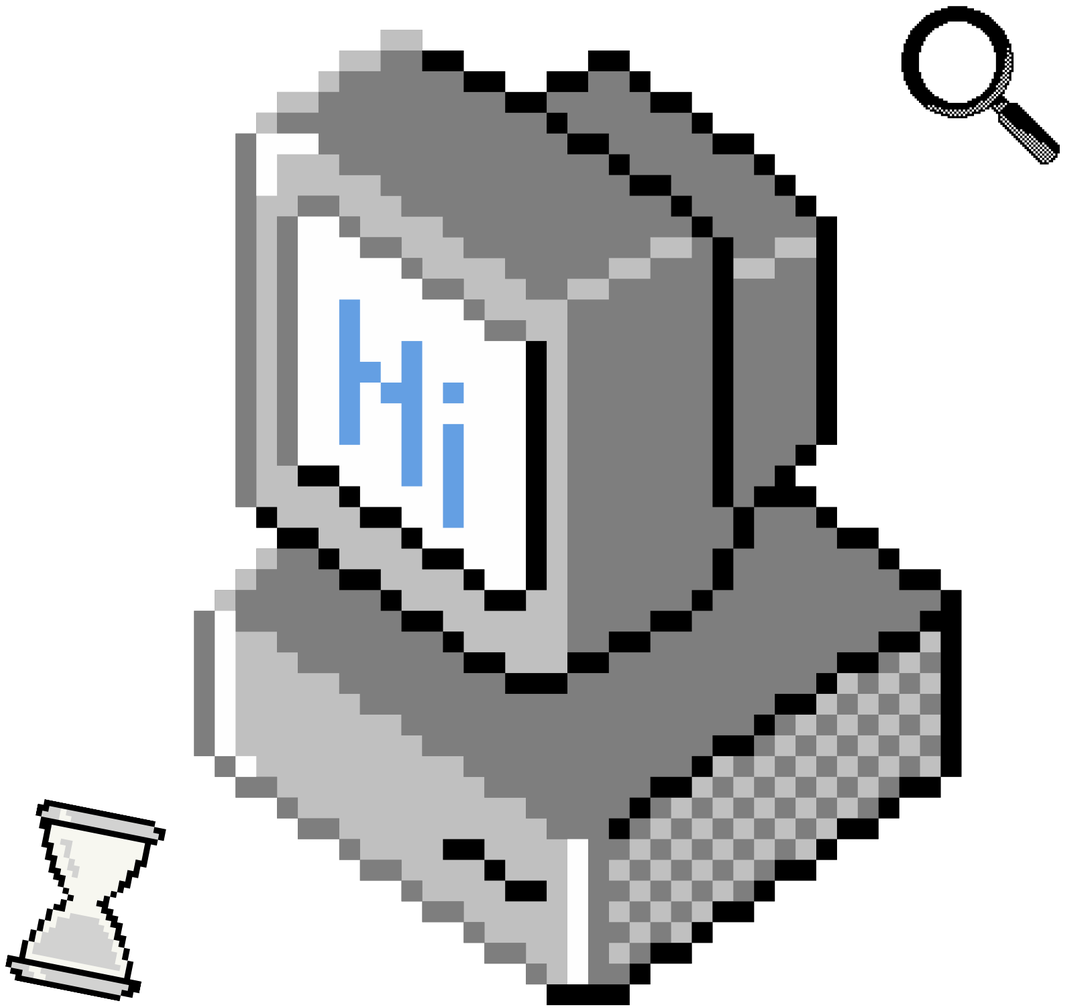


MERGE SORT

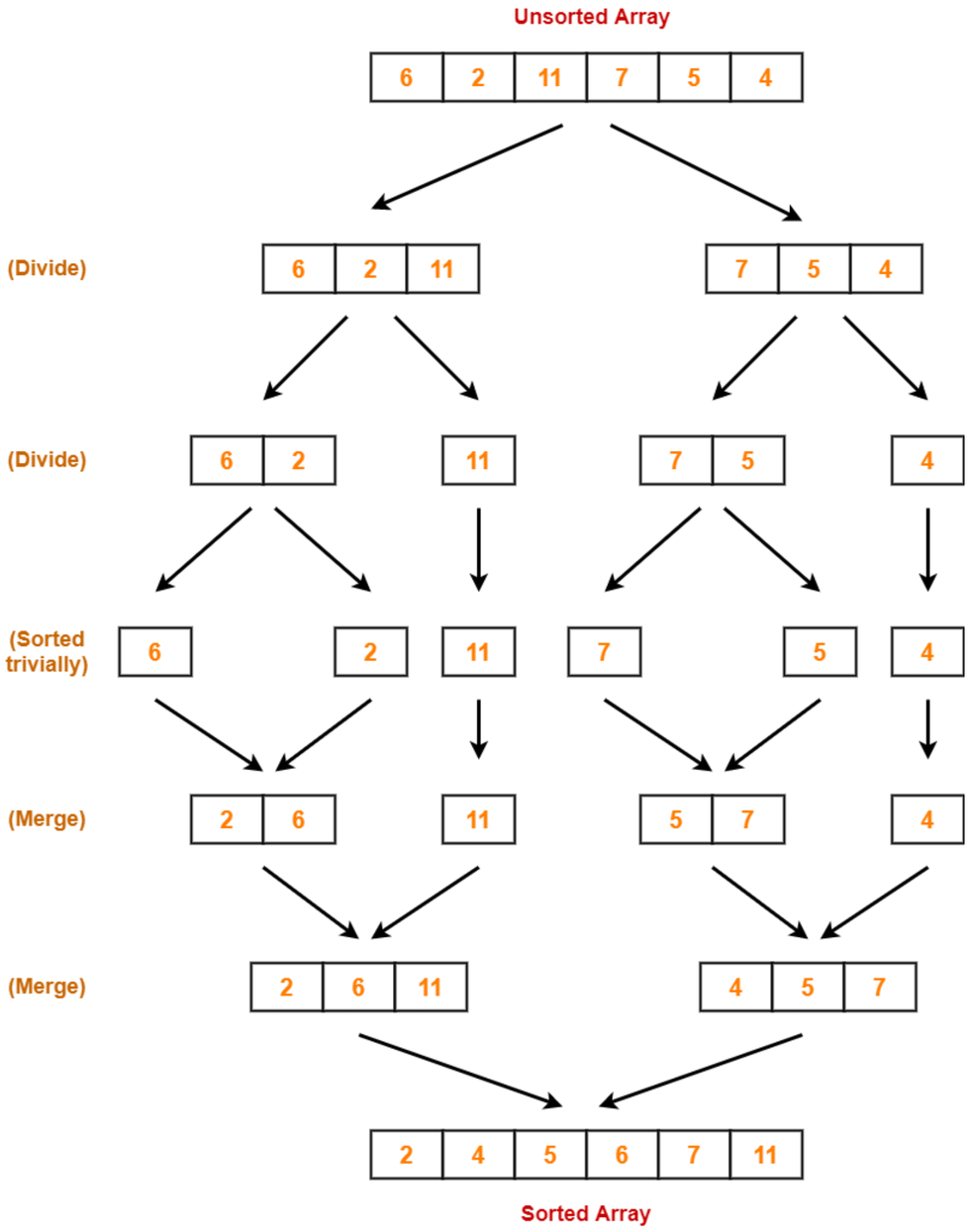
Sequencial VS Paralelo

Matheus Xavier e Nailson Chagas

O MERGE SORT

Merge Sort é um algoritmo de ordenação eficiente e estável baseado na estratégia Dividir e Conquistar

- **Dividir:** O array é dividido recursivamente em duas metades até não poder mais ser dividido (subarrays de tamanho 1).
- **Conquistar:** Cada subarray é ordenado individualmente (já estão ordenados quando possuem apenas 1 elemento).
- **Merge:** Os subarrays ordenados são combinados, formando arrays maiores também ordenados, até reconstruir o array completo.



COMO IMPLEMENTAMOS

Implementamos uma variação do Merge Sort que utiliza Insertion Sort para subarrays pequenos (≤ 64). Essa modificação foi adotada com o objetivo de melhorar o desempenho em arrays grandes e está sendo aplicada tanto na ***versão sequencial*** quanto na ***versão paralela*** do algoritmo.

```c
static void msort_seq(int *arr, int *tmp, int l, int r)
{
    int size = r - l + 1;

    if (size <= INSERTION_SORT_THRESHOLD)
    {
        insertion_sort(arr, l, r);
        return;
    }

    int m = (l + r) >> 1;

    msort_seq(arr, tmp, l, m);
    msort_seq(arr, tmp, m + 1, r);

    merge(arr, tmp, l, m, r);
}
```

AS VERSÕES SEQUENCIAIS

C

```c
static void msort_seq(int *arr, int *tmp, int l, int r)
{
    int size = r - l + 1;

    if (size <= INSERTION_SORT_THRESHOLD)
    {
        insertion_sort(arr, l, r);
        return;
    }

    int m = (l + r) >> 1;

    msort_seq(arr, tmp, l, m);
    msort_seq(arr, tmp, m + 1, r);

    merge(arr, tmp, l, m, r);
}
```

Java

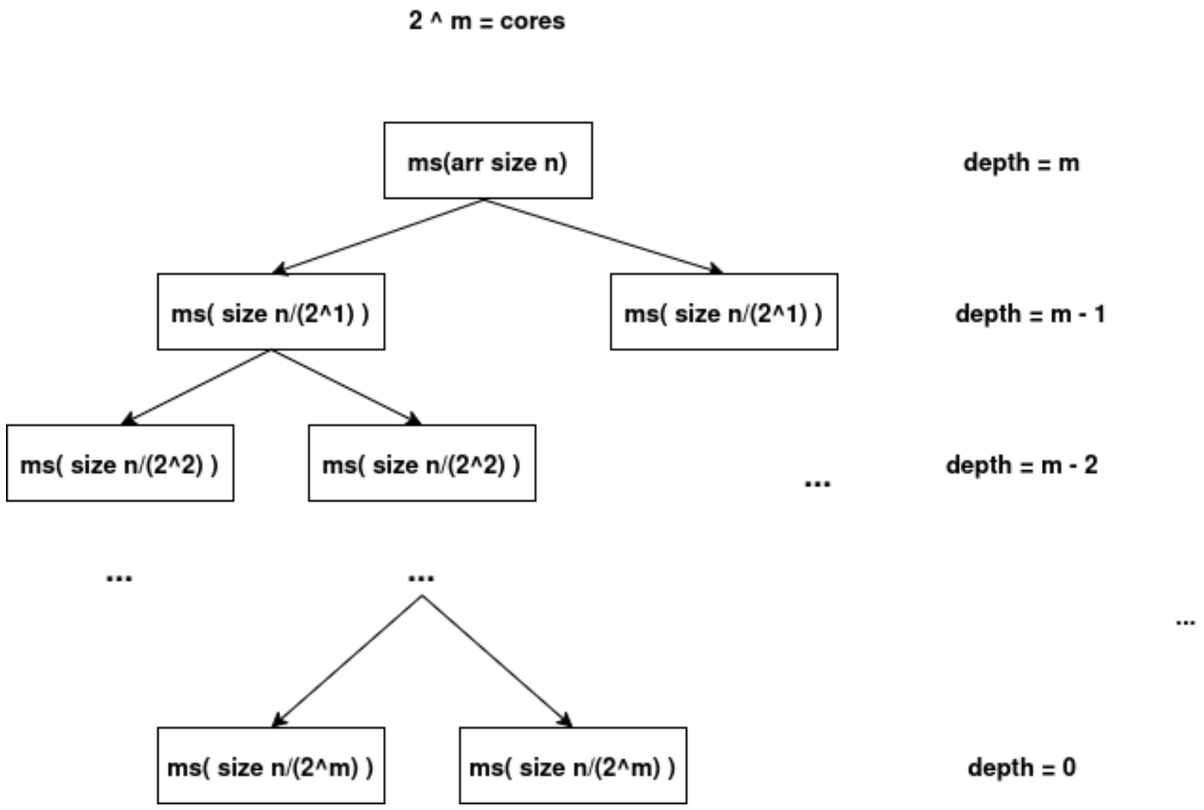
```java
private static void msort(int[] arr, int[] tmp, int l, int r) {
    int size = r - l + 1;

    if (size <= INSERTION_SORT_THRESHOLD) {
        insertionSort(arr, l, r);
        return;
    }

    int m = (l + r) / 2;

    msort(arr, tmp, l, m);
    msort(arr, tmp, m + 1, r);

    merge(arr, tmp, l, m, r);
}
```

AS VERSÕES PARALELAS

Uso natural do divisão e conquista para paralelismo, com cada divisão gerando duas threads

- ***PROBLEMA ENCONTRADO:*** Crescimento exponencial do número de threads
- ***SOLUÇÃO ESCOLHIDA:*** Limite na profundidade da árvore recursiva baseado no hardware disponível.

profundidade máxima = log2(número de núcleos)



AS VERSÕES PARALELAS

C - pthread

```c
static void msort_pt(int *arr, int *tmp, int l, int r, int depth)
{
    int size = r - l + 1;

    if (size <= INSERTION_SORT_THRESHOLD)
    {
        insertion_sort(arr, l, r);
        return;
    }

    int m = (l + r) >> 1;

    if (depth > 0)
    {
        pthread_t t1, t2;

        thread_args_t left = {arr, tmp, l, m, depth - 1};
        thread_args_t right = {arr, tmp, m + 1, r, depth - 1};

        pthread_create(&t1, NULL, thread_func, &left);
        pthread_create(&t2, NULL, thread_func, &right);

        pthread_join(t1, NULL);
        pthread_join(t2, NULL);
    }
    else
    {
        msort_pt(arr, tmp, l, m, 0);
        msort_pt(arr, tmp, m + 1, r, 0);
    }

    merge(arr, tmp, l, m, r);
}
```

Java - Threads

```java
private static void msort(int[] arr, int[] tmp, int l, int r, int depth) {
    int size = r - l + 1;

    if (size <= INSERTION_SORT_THRESHOLD) {
        insertionSort(arr, l, r);
        return;
    }

    int m = (l + r) / 2;

    if (depth > 0) {
        Thread t1 = new Thread(() -> msort(arr, tmp, l, m, depth - 1));
        Thread t2 = new Thread(() -> msort(arr, tmp, m + 1, r, depth - 1));

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    } else {
        msort(arr, tmp, l, m, 0);
        msort(arr, tmp, m + 1, r, 0);
    }

    merge(arr, tmp, l, m, r);
}
```

A VERSÃO PARALELA - OPENMP


```c
void msort(int *arr, int *tmp, int l, int r, int depth)
{
    int size = r - l + 1;

    if (size <= INSERTION_SORT_THRESHOLD)
    {
        insertion_sort(arr, l, r);
        return;
    }

    int m = (l + r) >> 1;

    if (depth > 0)
    {
#pragma omp task shared(arr, tmp)
        msort(arr, tmp, l, m, depth - 1);

#pragma omp task shared(arr, tmp)
        msort(arr, tmp, m + 1, r, depth - 1);

#pragma omp taskwait
    }
    else
    {
        msort(arr, tmp, l, m, 0);
        msort(arr, tmp, m + 1, r, 0);
    }

    merge(arr, tmp, l, m, r);
}
```

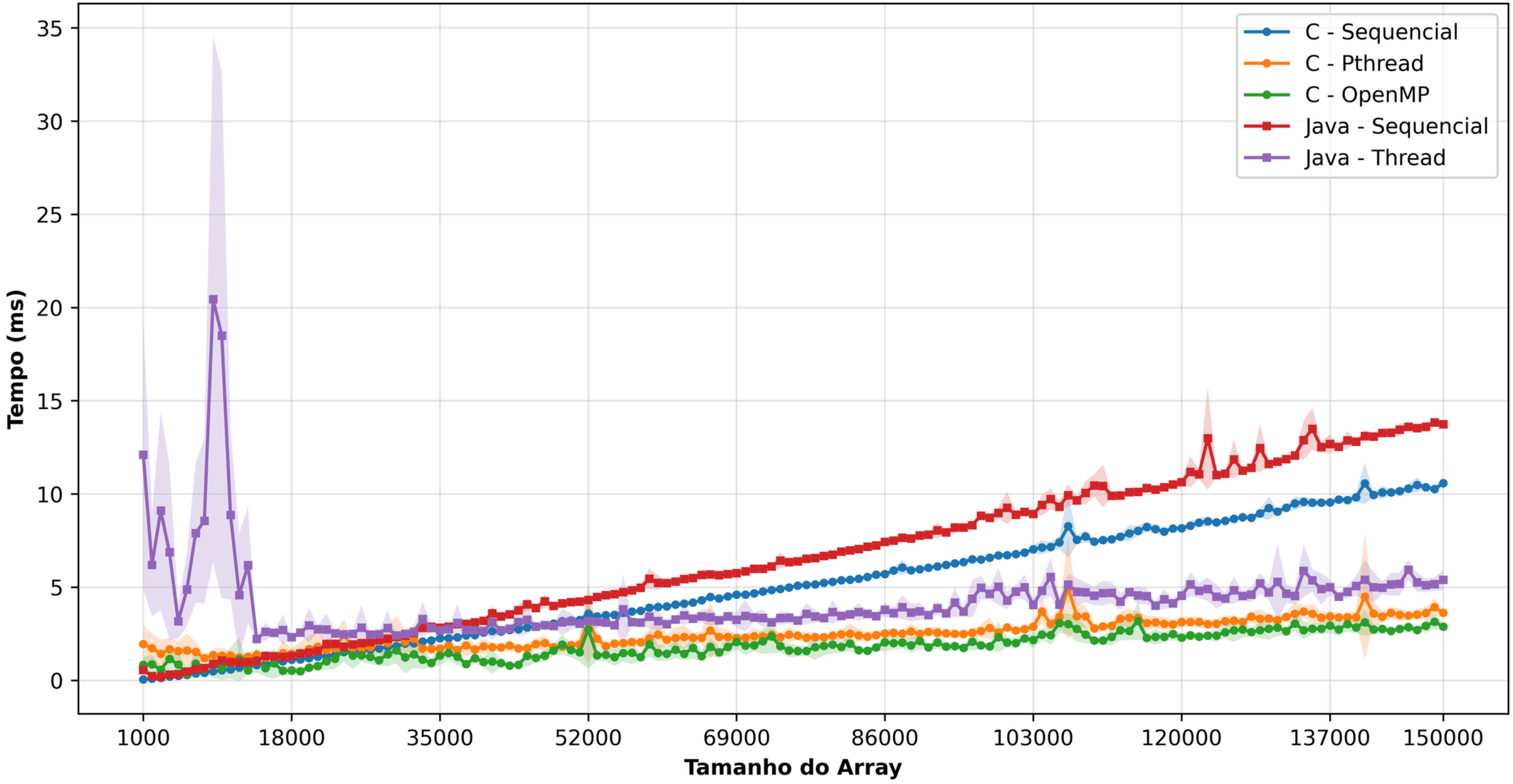
COMO TESTAMOS

- **Faixa de teste usada:**
  - Tamanhos de 1.000 até 150.000 elementos
  - Incremento de 1.000 elementos por iteração

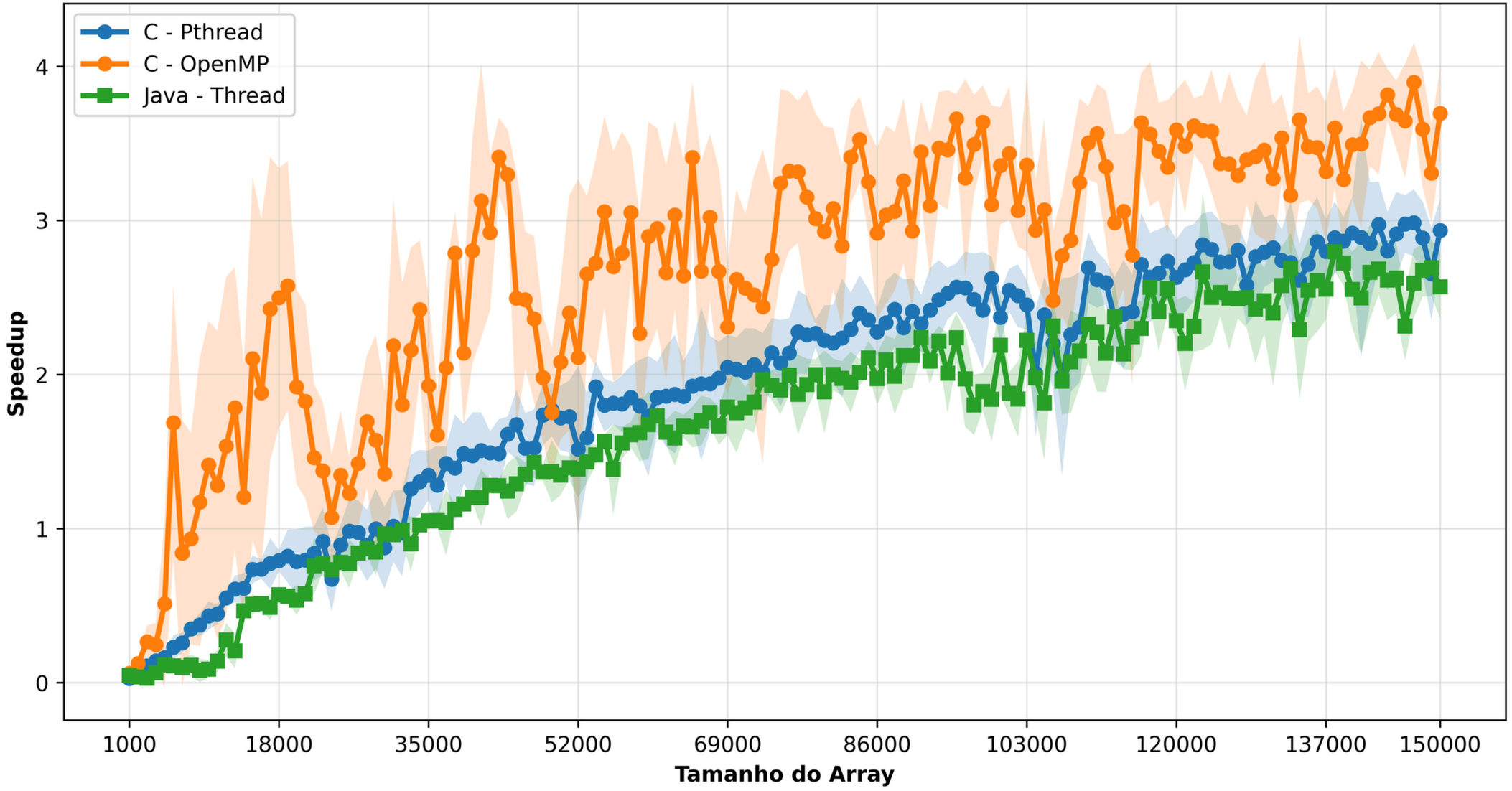
- **Especificação do computador usado:**
  - RAM: 64 64.0 GiB
  - Processador: AMD® Ryzen 5 2600 six-core processor × 12
  - OS: Pop!_OS 22.04 LTS

```
PARA cada tamanho de entrada:
    PARA 10 execuções:
        gerar array aleatório

        medir tempo de execução sequencial
        medir tempo de execução paralelo
        calcular speedup

    calcular média e desvio padrão
    salvar resultados no CSV
FIM
```

RESULTADOS - TEMPO MÉDIO



RESULTADOS - SPEEDUP



CONCLUSÕES

- **Desafios**
  - Profundidade do paralelismo: excesso de threads ↓ desempenho
  - Pthreads: código mais complexo e sujeito a erros
  - Entradas pequenas: alta variância (Java paralelo)

- **Melhor Implementação**
  - < 9.000 elementos: C sequencial (menos overhead)
  - Entradas grandes: C + OpenMP
    - Speedup médio: 3,22
    - Melhor em 140/150 casos